

Quarto Assignment - Loop Fusion

Generato da Doxygen 1.15.0

1 Gerarchia delle directory	1
1 Gerarchia delle directory	1
1.1 Directory	1
2 Indice dei namespace	2
2.1 Lista dei namespace	2
3 Indice della gerarchia	2
3.1 Gerarchia delle classi	2
4 Indice dei tipi composti	2
4.1 Elenco dei tipi composti	2
5 Indice dei file	2
5.1 Elenco dei file	2
6 Documentazione delle directory	3
6.1 Riferimenti per la directory test	3
7 Documentazione dei namespace	3
7.1 Riferimenti per il namespace anonymous_namespace{LoopFusion.cpp}	3
7.1.1 Documentazione delle funzioni	3
7.2 Riferimenti per il namespace llvm	7
8 Documentazione delle classi	7
8.1 Riferimenti per la struct llvm::LoopFusionPass	7
8.1.1 Documentazione delle funzioni membro	8
9 Documentazione dei file	9
9.1 Riferimenti per il file LoopFusion.cpp	9
9.1.1 Descrizione dettagliata	9
9.1.2 Documentazione delle funzioni	10
9.2 Riferimenti per il file LoopFusion.h	10
9.2.1 Descrizione dettagliata	11
9.3 LoopFusion.h	11
9.4 Riferimenti per il file test/loopFusion.cpp	12
9.4.1 Documentazione delle funzioni	12
Indice analitico	13

1 Gerarchia delle directory

1.1 Directory

test	3
loopFusion.cpp	12

2 Indice dei namespace

2.1 Lista dei namespace

Questa è l'elenco di tutti i namespace con una loro breve descrizione:

anonymous_namespace{LoopFusion.cpp}	3
llvm	7

3 Indice della gerarchia

3.1 Gerarchia delle classi

Questo elenco di ereditarietà è ordinato approssimativamente, ma non completamente, in ordine alfabetico:

PassInfoMixin	
llvm::LoopFusionPass	7

4 Indice dei tipi composti

4.1 Elenco dei tipi composti

Queste sono le classi, le struct, le union e le interfacce con una loro breve descrizione:

llvm::LoopFusionPass	7
--------------------------------------	---

5 Indice dei file

5.1 Elenco dei file

Questo è un elenco di tutti i file con una loro breve descrizione:

LoopFusion.cpp	
Implementazione del pass per la Loop Fusion	9
LoopFusion.h	
Header file del pass di Loop Fusion	10
test/loopFusion.cpp	12

6 Documentazione delle directory

6.1 Riferimenti per la directory test

File

- file [loopFusion.cpp](#)

7 Documentazione dei namespace

7.1 Riferimenti per il namespace `anonymous_namespace{LoopFusion.cpp}`

Funzioni

- static bool [isFusionCandidate](#) (Loop *L)
Funzione che controlla se un loop è adatto alla fusione.
- static void [sortCFESet](#) (SmallVector< Loop *, 4 > &Set, DominatorTree &DT)
Procedura che ordina i loop per dominanza.
- static bool [areAdjacent](#) (Loop *L0, Loop *L1)
Funzione che verifica l'adiacenza tra due loop.
- static bool [haveLoopTripCount](#) (Loop *L0, Loop *L1, ScalarEvolution &SE)
Funzione che confronta il loop trip count di due loop utilizzando la Scalar Evolution.
- static bool [areControlFlowEq](#) (Loop *L0, Loop *L1, DominatorTree &DT, PostDominatorTree &PDT)
Funzione che determina se due loop sono control flow equivalenti, ovvero se L0 domina L1, allora L1 postdomina L0.
- static bool [dependenceAnalysis](#) (Loop *L0, Loop *L1, DependenceInfo &DI)
Funzione che analizza le dipendenze.
- static BasicBlock * [getLoopBodyEntry](#) (Loop *L)
Funzione che trova il primo blocco del corpo di un loop.
- static Loop * [fuseLoops](#) (Loop *L0, Loop *L1, LoopInfo &LI, DominatorTree &DT)
Funzione che fonde i due loop e ritorna il loop fuso.

7.1.1 Documentazione delle funzioni

`areAdjacent()`

```
bool anonymous_namespace{LoopFusion.cpp}::areAdjacent (
    Loop * L0,
    Loop * L1) [static]
```

Funzione che verifica l'adiacenza tra due loop.

Se i loop sono guarded, il successore non loop del guard branch del primo loop deve essere l'entry block dell'altro. Questo si verifica ottenendo i blocchi successori del guard branch e controllando se fanno parte del loop. Appena troviamo il successore che non appartiene al loop, verifichiamo se è uguale al preheader del secondo loop. Se i loop sono unguarded l'exit block del primo loop deve essere il preheader del secondo loop.

Parametri

<i>L1</i>	
<i>L2</i>	

Restituisce

true se sono adiacenti, false altrimenti

areControlFlowEq()

```
bool anonymous_namespace{LoopFusion.cpp}::areControlFlowEq (  
    Loop * L0,  
    Loop * L1,  
    DominatorTree & DT,  
    PostDominatorTree & PDT) [static]
```

Funzione che determina se due loop sono control flow equivalenti, ovvero se *L0* domina *L1*, allora *L1* postdomina *L0*.

Parametri

<i>L0</i>	
<i>L1</i>	
<i>DT</i>	
<i>PDT</i>	

Restituisce

true se *L0* e *L1* sono CFE, false altrimenti

dependenceAnalysis()

```
bool anonymous_namespace{LoopFusion.cpp}::dependenceAnalysis (  
    Loop * L0,  
    Loop * L1,  
    DependenceInfo & DI) [static]
```

Funzione che analizza le dipendenze.

Parametri

<i>L0</i>	
<i>L1</i>	
<i>DI</i>	

Restituisce

true se le dipendenze sono valide, false altrimenti

fuseLoops()

```
Loop * anonymous_namespace{LoopFusion.cpp}::fuseLoops (
    Loop * L0,
    Loop * L1,
    LoopInfo & LI,
    DominatorTree & DT) [static]
```

Funzione che fonde i due loop e ritorna il loop fuso.

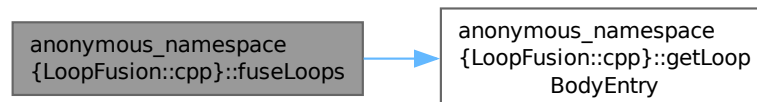
Parametri

<i>L0</i>	
<i>L1</i>	
<i>LI</i>	
<i>DT</i>	

Restituisce

L0 fuso

Questo è il grafo delle chiamate per questa funzione:

**getLoopBodyEntry()**

```
BasicBlock * anonymous_namespace{LoopFusion.cpp}::getLoopBodyEntry (
    Loop * L) [static]
```

Funzione che trova il primo blocco del corpo di un loop.

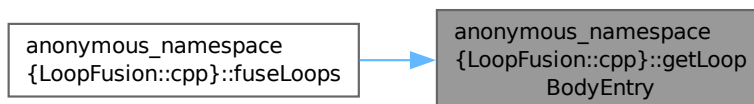
Parametri

<i>L</i>	
----------	--

Restituisce

Succ se trova il successore, nullptr altrimenti

Questo è il grafo dei chiamanti di questa funzione:

**haveIdloopTripCount()**

```

bool anonymous_namespace{LoopFusion.cpp}::haveIdloopTripCount (
    Loop * L0,
    Loop * L1,
    ScalarEvolution & SE) [static]
  
```

Funzione che confronta il loop trip count di due loop utilizzando la Scalar Evolution.

Parametri

<i>L0</i>	
<i>L1</i>	
<i>SE</i>	

Restituisce

true se hanno loop trip count identico, false altrimenti

isFusionCandidate()

```

bool anonymous_namespace{LoopFusion.cpp}::isFusionCandidate (
    Loop * L) [static]
  
```

Funzione che controlla se un loop è adatto alla fusione.

Per essere fuso, un loop deve essere in forma normale, deve possedere tutti i blocchi importanti come il preheader, header, latch ed exiting blocks. Inoltre non deve lanciare eccezioni e non deve contenere accessi volatili sulla memoria.

Parametri

<i>L</i>	Loop da controllare
----------	---------------------

Restituisce

true se il loop *L* è candidato per la fusione, false altrimenti

sortCFESet()

```
void anonymous_namespace{LoopFusion.cpp}::sortCFESet (
    SmallVector< Loop *, 4 > & Set,
    DominatorTree & DT) [static]
```

Procedura che ordina i loop per dominanza.

Parametri

<i>Set</i>	il set di loop che sono Contol Flow Equivalenti
<i>DT</i>	Dominator tree

7.2 Riferimenti per il namespace llvm**Composti**

- struct [LoopFusionPass](#)

8 Documentazione delle classi**8.1 Riferimenti per la struct llvm::LoopFusionPass**

```
#include <LoopFusion.h>
```

Diagramma delle classi per llvm::LoopFusionPass

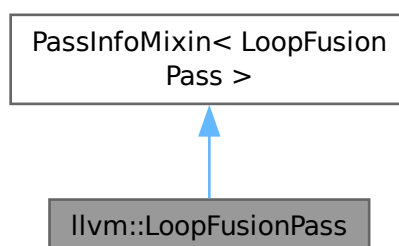
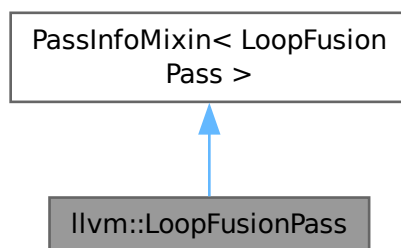


Diagramma di collaborazione per `llvm::LoopFusionPass`:



Membri pubblici

- `PreservedAnalyses run` (`Function &F`, `FunctionAnalysisManager &AM`)
Function pass che effettua tutte le trasformazioni sui loop.

8.1.1 Documentazione delle funzioni membro

`run()`

```
PreservedAnalyses LoopFusionPass::run (  
    Function & F,  
    FunctionAnalysisManager & AM)
```

Function pass che effettua tutte le trasformazioni sui loop.

Parametri

<i>F</i>	Funzione su cui applicare i passi di trasformazione
<i>AM</i>	New pass manager per funzioni

Restituisce

Un oggetto `PreservedAnalyses` che indica quali analisi sono rimaste valide dopo il pass

La documentazione per questa struct è stata generata a partire dai seguenti file:

- [LoopFusion.h](#)
- [LoopFusion.cpp](#)

9 Documentazione dei file

9.1 Riferimenti per il file LoopFusion.cpp

Implementazione del pass per la Loop Fusion.

```
#include "LoopFusion.h"
```

Grafo delle dipendenze di inclusione per LoopFusion.cpp:



Namespace

- namespace `anonymous_namespace{LoopFusion.cpp}`

Funzioni

- static bool `anonymous_namespace{LoopFusion.cpp}::isFusionCandidate` (Loop *L)
Funzione che controlla se un loop è adatto alla fusione.
- static void `anonymous_namespace{LoopFusion.cpp}::sortCFESet` (SmallVector< Loop *, 4 > &Set, DominatorTree &DT)
Procedura che ordina i loop per dominanza.
- static bool `anonymous_namespace{LoopFusion.cpp}::areAdjacent` (Loop *L0, Loop *L1)
Funzione che verifica l'adiacenza tra due loop.
- static bool `anonymous_namespace{LoopFusion.cpp}::haveLDLoopTripCount` (Loop *L0, Loop *L1, ScalarEvolution &SE)
Funzione che confronta il loop trip count di due loop utilizzando la Scalar Evolution.
- static bool `anonymous_namespace{LoopFusion.cpp}::areControlFlowEq` (Loop *L0, Loop *L1, DominatorTree &DT, PostDominatorTree &PDT)
Funzione che determina se due loop sono control flow equivalenti, ovvero se L0 domina L1, allora L1 postdomina L0.
- static bool `anonymous_namespace{LoopFusion.cpp}::dependenceAnalysis` (Loop *L0, Loop *L1, DependenceInfo &DI)
Funzione che analizza le dipendenze.
- static BasicBlock * `anonymous_namespace{LoopFusion.cpp}::getLoopBodyEntry` (Loop *L)
Funzione che trova il primo blocco del corpo di un loop.
- static Loop * `anonymous_namespace{LoopFusion.cpp}::fuseLoops` (Loop *L0, Loop *L1, LoopInfo &LI, DominatorTree &DT)
Funzione che fonde i due loop e ritorna il loop fuso.
- llvm::PassPluginLibraryInfo `getLoopFusionPassPluginInfo` ()
- LLVM_ATTRIBUTE_WEAK::llvm::PassPluginLibraryInfo `llvmGetPassPluginInfo` ()

9.1.1 Descrizione dettagliata

Implementazione del pass per la Loop Fusion.

9.1.2 Documentazione delle funzioni

getLoopFusionPassPluginInfo()

```
llvm::PassPluginLibraryInfo getLoopFusionPassPluginInfo ()
```

Questo è il grafo dei chiamanti di questa funzione:



llvmGetPassPluginInfo()

```
LLVM_ATTRIBUTE_WEAK::llvm::PassPluginLibraryInfo llvmGetPassPluginInfo ()
```

Questo è il grafo delle chiamate per questa funzione:



9.2 Riferimenti per il file LoopFusion.h

Header file del pass di Loop Fusion.

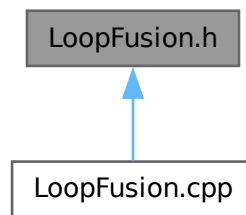
```
#include "llvm/IR/Function.h"
#include "llvm/IR/Instructions.h"
#include "llvm/IR/Constants.h"
#include "llvm/Passes/PassBuilder.h"
#include "llvm/Passes/PassPlugin.h"
#include "llvm/Support/raw_ostream.h"
#include "llvm/Analysis/LoopInfo.h"
#include "llvm/IR/Dominators.h"
#include "llvm/Analysis/PostDominators.h"
#include "llvm/ADT/DepthFirstIterator.h"
#include "llvm/Analysis/ScalarEvolution.h"
#include "llvm/Analysis/DependenceAnalysis.h"
```

```
#include "llvm/Transforms/Utils/BasicBlockUtils.h"
```

Grafo delle dipendenze di inclusione per LoopFusion.h:



Questo grafo mostra quali altri file includono direttamente o indirettamente questo file:



Composti

- struct `llvm::LoopFusionPass`

Namespace

- namespace `llvm`

9.2.1 Descrizione dettagliata

Header file del pass di Loop Fusion.

9.3 LoopFusion.h

[Vai alla documentazione di questo file.](#)

```
00001
00005
00006 #ifndef TRANSFORM_LOOPFUSION_H
00007 #define TRANSFORM_LOOPFUSION_H
00008
00009 #include "llvm/IR/Function.h"
00010 #include "llvm/IR/Instructions.h"
00011 #include "llvm/IR/Constants.h"
00012
00013 #include "llvm/Passes/PassBuilder.h"
00014 #include "llvm/Passes/PassPlugin.h"
00015 #include "llvm/Support/raw_ostream.h"
00016
00017 #include "llvm/Analysis/LoopInfo.h"
00018 #include "llvm/IR/Dominators.h"
00019 #include "llvm/Analysis/PostDominators.h"
00020 #include "llvm/ADT/DepthFirstIterator.h"
```

```
00021 #include "llvm/Analysis/ScalarEvolution.h"
00022 #include "llvm/Analysis/DependenceAnalysis.h"
00023 #include "llvm/Transforms/Utils/BasicBlockUtils.h"
00024
00025
00026 namespace llvm{
00027
00028     struct LoopFusionPass : PassInfoMixin<LoopFusionPass> {
00029         PreservedAnalyses run(Function &F, FunctionAnalysisManager &AM);
00030     };
00031 }
00032
00033 #endif
```

9.4 Riferimenti per il file test/loopFusion.cpp

Funzioni

- int [LICM](#) (int a, int b)

9.4.1 Documentazione delle funzioni

LICM()

```
int LICM (
    int a,
    int b)
```

Indice analitico

- anonymous_namespace{LoopFusion.cpp}, [3](#)
 - areAdjacent, [3](#)
 - areControlFlowEq, [4](#)
 - dependenceAnalysis, [4](#)
 - fuseLoops, [4](#)
 - getLoopBodyEntry, [5](#)
 - haveIdloopTripCount, [6](#)
 - isFusionCandidate, [6](#)
 - sortCFESet, [7](#)
- areAdjacent
 - anonymous_namespace{LoopFusion.cpp}, [3](#)
- areControlFlowEq
 - anonymous_namespace{LoopFusion.cpp}, [4](#)
- dependenceAnalysis
 - anonymous_namespace{LoopFusion.cpp}, [4](#)
- fuseLoops
 - anonymous_namespace{LoopFusion.cpp}, [4](#)
- getLoopBodyEntry
 - anonymous_namespace{LoopFusion.cpp}, [5](#)
- getLoopFusionPassPluginInfo
 - LoopFusion.cpp, [10](#)
- haveIdloopTripCount
 - anonymous_namespace{LoopFusion.cpp}, [6](#)
- isFusionCandidate
 - anonymous_namespace{LoopFusion.cpp}, [6](#)
- LICM
 - loopFusion.cpp, [12](#)
- llvm, [7](#)
- llvm::LoopFusionPass, [7](#)
 - run, [8](#)
- llvmGetPassPluginInfo
 - LoopFusion.cpp, [10](#)
- LoopFusion.cpp, [9](#)
 - getLoopFusionPassPluginInfo, [10](#)
 - llvmGetPassPluginInfo, [10](#)
- loopFusion.cpp
 - LICM, [12](#)
- LoopFusion.h, [10](#)
- Riferimenti per la directory test, [3](#)
- run
 - llvm::LoopFusionPass, [8](#)
- sortCFESet
 - anonymous_namespace{LoopFusion.cpp}, [7](#)
- test/loopFusion.cpp, [12](#)